M:N User-to-Kernel Level Multithreading Library

Pranjal Kabra

April 17, 2025

Contents

1 Introduction

1.1 What is a Thread?

A thread is the smallest unit of execution within a process. Multiple threads can run within a single process and share its resources.

1.2 What is Multithreading?

Multithreading in operating systems refers to the ability of a process to execute multiple threads concurrently. This allows for better resource utilization and parallelism.

2 Types of Threads

- **User-Level Threads (ULTs)**: Managed by user-level libraries without kernel involvement. These have faster context switching but a blocking system call from any thread blocks the entire process.
- **Kernel-Level Threads (KLTs)**: Managed directly by the operating system. They allow true parallelism but have higher overhead due to kernel involvement.

2.1 Multithreading Models

- **Many-to-One**: Maps many user threads to a single kernel thread. Simple, but lacks parallel execution.
- **One-to-One**: Maps each user thread to its own kernel thread. Provides concurrency but incurs greater overhead.
- **Many-to-Many (M:N)**: Maps many user threads to a smaller or equal number of kernel threads. Balances flexibility and performance.

3 About This Project

This library implements an M:N user-to-kernel level threading model, offering efficient thread management with minimal performance overhead. It supports custom scheduling and better concurrency for blocking operations.

3.1 Motivation

The M:N model provides a middle ground—reduced context switching overhead, enhanced scheduling control, and efficient use of system resources.

4 Key Features

- Initialization of custom number of kernel and user threads
- User and kernel thread creation
- Preemptive thread yielding within the same kernel thread

- Thread synchronization using `mn_thread_wait()`
- Visual mapping of user threads to kernel threads
- Two test cases: HTTP simulation and factorial calculation

5 Library Structure

```
mn_thread_lib/
         include/
              mn_thread.h        # Thread definitions
         build/                 # Compiled binaries
         src/
              mn_thread.c        # Core implementation
              scheduler.c        # Scheduler logic
         test/
              test_http_sim.c    # HTTP simulation test
              test_fact.c        # Factorial calculation test
         Makefile
```

6 Building and Running the Project

6.1 Prerequisites

- Linux OS
- GCC compiler
- `make` utility

6.2 Cloning and Building

```
git clone https://github.com/PranjalKabra/M_userT-N_kernelT-multithreading-library.git
cd mn_thread_lib
make
```

6.3 Running Tests

6.3.1 HTTP Simulation

```
make http_sim
```

Description: Simulates 16 user threads fetching web pages, mapped across 4 kernel threads using Round Robin scheduling.

6.3.2 Factorial Calculation

```
make test_fact
```

Description: Calculates 15! using 6 user threads and 3 kernel threads. You can modify `N` in `test_fact.c`.

6.4 Cleaning Build Files

```
make clean
```

7 Test Case Analysis

7.1 HTTP Simulation

The test simulates a scenario where 16 user threads issue HTTP requests, and 4 kernel threads act as servers. Each kernel thread handles 4 user threads via Round Robin.

Example Mapping:

- KT0 → U0, U4, U8, U12
- KT1 → U1, U5, U9, U13
- ...

Each user thread has a burst time of 12 units, and the time quantum is 5 units. Threads are preempted accordingly at remaining times 7 and 2, then exit when completed.

7.2 Factorial Calculation

This test divides the computation of 15! among 3 kernel threads and 6 user threads:

- K0 → [1–5]
- K1 → [6–10]
- K2 → [11–15]

Each kernel thread spawns 2 user threads to handle subranges (e.g., K0: U0 → [1–3], U3 → [4–5]). The results are multiplied together at the end to get the final output.

Note: `mn_thread_wait()` ensures all user threads terminate before their kernel thread concludes, ensuring correctness.

8 Conclusion

This project demonstrates the effectiveness of the M:N threading model by showing performance and concurrency improvements. It supports customizable threading, cooperative scheduling, and modular design, making it a solid foundation for multithreading systems.